

# Relatório Técnico - UrbanFlow

**Sistema de Simulação e Gestão Inteligente de Tráfego Urbano**

Licenciatura em Engenharia Informática

Projeto Final de Curso

Autora: Beatriz Antunes

Equipa Orientação: Frederico branco

Coorientação: João Paulo Moura, Emanuel Peres

Data: 13 Maio 2026

## Resumo

O **UrbanFlow** é uma plataforma web de simulação e análise de tráfego urbano desenvolvida como projeto final de curso de Licenciatura em Engenharia Informática. O sistema combina um motor de simulação em JavaScript, inteligência artificial adaptativa, aprendizagem por reforço (Q-Learning) e um motor de regras em linguagem natural para modelar e comparar estratégias de controlo semafórico em ambiente urbano.

A plataforma integra dados reais de transporte público português através do formato GTFS (18 distritos) e disponibiliza múltiplas vistas analíticas, métricas em tempo real, comparação AI vs. controlo tradicional, análise ambiental e de segurança, e histórico de simulações. Os resultados experimentais demonstram reduções consistentes de 40–55% no tempo médio de espera e de 25–35% nas emissões de CO<sub>2</sub> face ao controlo de ciclo fixo, validando a eficácia do controlo adaptativo em cenários urbanos.

A plataforma encontra-se publicamente implantada em ambiente cloud (frontend em Vercel, backend em Render), recorrendo exclusivamente a serviços gratuitos.

**Palavras-chave:** simulação de tráfego, controlo semafórico adaptativo, aprendizagem por reforço, processamento de linguagem natural, GTFS, emissões de CO<sub>2</sub>, React, Python

## Abstract

**UrbanFlow** is a web-based urban traffic simulation and analysis platform developed as a final-year Computer Engineering project. The system combines a JavaScript simulation engine, adaptive artificial intelligence, reinforcement learning (Q-Learning), and a natural language rule engine to model and compare traffic signal control strategies in an urban environment. The platform integrates real Portuguese public transport data via the GTFS format (18 districts) and provides multiple analytical views, real-time metrics, AI vs. traditional control comparison, environmental and safety analysis, and simulation history. Experimental results consistently show reductions of 40–55% in average vehicle wait time and 25–35% in CO<sub>2</sub> emissions compared to fixed-cycle control, validating the effectiveness of adaptive control in urban scenarios.

The platform is publicly deployed in a cloud environment (frontend on Vercel, backend on Render), using exclusively free-tier services.

**Keywords:** traffic simulation, adaptive traffic signal control, reinforcement learning, natural language processing, GTFS, CO<sub>2</sub> emissions, React, Python

# Índice

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Relatório Técnico - UrbanFlow .....                                 | 1  |
| Resumo .....                                                        | 2  |
| Abstract .....                                                      | 2  |
| 1.Introdução .....                                                  | 5  |
| 1.1 Contextualização .....                                          | 5  |
| 1.2 Objetivo Principal .....                                        | 6  |
| 1.3 Objetivos Específicos .....                                     | 6  |
| 2. Investigação e Estado da Arte .....                              | 7  |
| 2.1 Revisão de Literatura .....                                     | 7  |
| 2.1.1 Controlo Semafórico Clássico e Adaptativo .....               | 7  |
| 2.1.2 Aprendizagem por Reforço Aplicada a Semáforos .....           | 7  |
| 2.1.3 Processamento de Linguagem Natural para Tráfego .....         | 8  |
| 2.2 Análise de Soluções Existentes .....                            | 8  |
| 2.3 Seleção Tecnológica .....                                       | 9  |
| 3. Engenharia de Requisitos e Modelação .....                       | 10 |
| 3.1 Levantamento de Requisitos .....                                | 10 |
| 3.2. Modelação Funcional(UML) .....                                 | 13 |
| 3.2.1 Atores do Sistema .....                                       | 13 |
| 3.2.2 Diagramas de Casos de Uso .....                               | 14 |
| 3.3 Modelação Comportamental (UML) .....                            | 21 |
| 3.3.1 Diagrama de Sequência - Aplicar Regra de Tráfego (UC03) ..... | 21 |
| 3.3.2 Diagrama de Atividades - Ciclo Principal de Simulação .....   | 22 |
| 3.3.3 Diagrama de Estados - Semáforo .....                          | 23 |
| 3.4. Modelação de Dados (UML/DER) .....                             | 25 |
| 3.4.1 Diagrama de Estados – Semáforo .....                          | 25 |
| 4. Setup do Ambiente e Ferramentas .....                            | 26 |
| 5. Planeamento e Desenvolvimento Incremental .....                  | 27 |
| 6.Planeamento e Desenvolvimento Incremental .....                   | 28 |
| 6.1 Software Funcional .....                                        | 28 |
| 6.2 Repositório de Código .....                                     | 31 |
| 6.3 Base de Conhecimento e Workspace .....                          | 31 |
| 6.4 Discussão Crítica do Fluxo de Vibe Coding .....                 | 32 |
| 7. Conclusão e Trabalho Futuro .....                                | 34 |
| 7.1 Sumário do Projeto .....                                        | 34 |
| 7.2 Limitações .....                                                | 34 |
| 7.3 Trabalho Futuro .....                                           | 34 |
| 19. Referências .....                                               | 35 |

## Índice de Imagens

|                                  |    |
|----------------------------------|----|
| Figure 1 - Página Overview ..... | 29 |
| Figure 2 - Página LiveMaps ..... | 29 |
| Figure 3 - GTFS Map.....         | 30 |
| Figure 4 - Analytics .....       | 30 |
| Figure 5 - Página Reports .....  | 31 |

# 1.Introdução

## 1.1 Contextualização

A gestão do tráfego urbano constitui um dos desafios centrais das cidades contemporâneas. O aumento da motorização individual, a multiplicidade de modos de transporte e a necessidade de reduzir emissões de gases com efeito de estufa colocam pressão crescente sobre as infraestruturas viárias e sobre os sistemas de controlo semafórico tradicionais, baseados em ciclos de tempo fixo. Esses sistemas, dimensionados para um cenário médio, são incapazes de responder dinamicamente a variações reais de fluxo, horas de ponta, acidentes, eventos pontuais, presença de veículos de emergência, gerando ineficiência, congestionamento e emissões evitáveis.

Paralelamente, a disponibilidade de dados abertos de transporte público (nomeadamente através do formato *General Transit Feed Specification*, GTFS) e a maturidade de técnicas de inteligência artificial em particular aprendizagem por reforço e processamento de linguagem natural abrem novas possibilidades para sistemas adaptativos de gestão de tráfego que respondem em tempo real às condições reais da rede. No entanto, as ferramentas profissionais existentes (SUMO, VISSIM, Aimsun) apresentam custo elevado, complexidade de configuração e ausência de interfaces acessíveis, tornando-as inadequadas para contextos educativos, de prototipagem rápida ou de demonstração quantitativa do impacto da IA no tráfego urbano.

O presente projeto situa-se nesta lacuna: propõe-se desenvolver uma plataforma web acessível, executável no *browser*, que combine simulação multiagente, controlo adaptativo por IA, aprendizagem por reforço e regras em linguagem natural, com integração de dados reais portugueses.

## 1.2 Objetivo Principal

Desenvolver uma plataforma web de simulação de tráfego urbano que demonstre, de forma quantitativa e visualmente intuitiva, as vantagens do controlo semafórico adaptativo (heurístico e por aprendizagem por reforço) face ao controlo tradicional de ciclo fixo, recorrendo a dados reais de transporte público português e disponibilizada publicamente em ambiente *cloud* gratuito.

## 1.3 Objetivos Específicos

- Simular o comportamento de tráfego urbano sob diferentes cenários (normal, hora de ponta, acidente, emergência) e modos de controlo (Tradicional, IA Heurística, *Reinforcement Learning*).
- Comparar quantitativamente, em tempo real e com a mesma semente aleatória, o controlo por IA e o controlo tradicional, recorrendo a uma *shadow engine* de execução paralela.
- Integrar dados reais de redes de transporte público portuguesas (GTFS de 18 distritos) para calibrar perfis horários de *spawn* de veículos.
- Permitir a definição de regras operacionais em linguagem natural portuguesa (ex.: “Priorizar ambulâncias na hora de ponta”).
- Registar e analisar o histórico de simulações, com persistência local entre sessões e replicação opcional em base de dados.
- Implantar publicamente a plataforma em ambiente *cloud* gratuito, demonstrando a viabilidade de soluções de baixo custo neste domínio.
- Aplicar e documentar criticamente uma metodologia de desenvolvimento assistida por IA generativa (*Vibe Coding*), tomando a modelação UML rigorosa como contexto-base das *prompts*.

## 2. Investigação e Estado da Arte

### 2.1 Revisão de Literatura

A revisão de literatura concentra-se em três eixos temáticos que sustentam as decisões técnicas do UrbanFlow: o controlo semafórico clássico, a aprendizagem por reforço aplicada a semáforos e o processamento de linguagem natural para sistemas operacionais de tráfego.

#### 2.1.1 Controlo Semafórico Clássico e Adaptativo

A formalização matemática da otimização semafórica remonta a Webster (1958), cuja fórmula clássica define a duração ótima de ciclo em função do fluxo observado e ainda é usada como referência em simuladores modernos. Sobre esta base desenvolveram-se sistemas adaptativos em produção, nomeadamente o **SCOOT** (*Split Cycle Offset Optimisation Technique*) e o **SCATS**, sistemas centralizados implantados em cidades reais que requerem infraestrutura de sensores dedicada e, por isso, não são acessíveis para experimentação académica.

Diakaki, Papageorgiou e Aboudolas (2002) propuseram uma abordagem multivariável baseada em controlador regulador linear, demonstrando ganhos de 10–20% em *throughput* em redes urbanas, mas a calibração requerida por cenário limita a sua aplicabilidade fora do contexto experimental original.

#### 2.1.2 Aprendizagem por Reforço Aplicada a Semáforos

A aplicação de *Reinforcement Learning* (RL) ao controlo de semáforos tornou-se uma linha de investigação ativa a partir de 2018, com o trabalho seminal de Wei, Zheng, Yao e Li (2018), **IntelliLight**: primeira aplicação de *deep RL* a semáforos urbanos com dados reais, recorrendo a Deep Q-Networks (DQN). A principal limitação do IntelliLight é a necessidade de GPU e a baixa reprodutibilidade fora do *setup* original.

Wei et al. (2019) desenvolveram **PressLight**, introduzindo o conceito de função de recompensa baseada em “pressão” de fila, que generaliza melhor entre topologias e densidades de tráfego. Chen et al. (2020) propuseram **MPLight**, com partilha de parâmetros entre agentes para coordenar eficientemente múltiplos semáforos, mas com dependência do simulador externo CityFlow.

A fundamentação teórica do Q-Learning utilizado neste projeto encontra-se em Watkins e Dayan (1992), que provaram a convergência do algoritmo para ambientes markovianos finitos, e na obra de referência de Sutton e Barto (2018), que sistematiza o quadro conceptual da aprendizagem por reforço moderna.

### 2.1.3 Processamento de Linguagem Natural para Tráfego

A interface em linguagem natural para configuração de sistemas operacionais de tráfego é uma área emergente com poucos trabalhos diretos. Estudos existentes focam-se maioritariamente em *chatbots* de informação ao utilizador de transporte público — não aplicável a controlo operacional. Trabalhos recentes (2023–2024) exploram a utilização de modelos de linguagem de grande escala (LLMs) para gerar configurações de simulação a partir de descrições textuais, mas a dependência de APIs externas e a latência associada tornam estas soluções inadequadas para uso em tempo real.

## 2.2 Análise de Soluções Existentes

A análise comparativa de soluções existentes foi conduzida em dois domínios: ferramentas de simulação de tráfego e plataformas de exploração de dados GTFS. O Quadro 2.1 sintetiza as ferramentas de simulação mais estabelecidas e as suas limitações face ao âmbito do UrbanFlow.

| Ferramenta                                 | Tipo                 | Modelo                     | Limitações face ao TrafficAI                                                                                 |
|--------------------------------------------|----------------------|----------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>SUMO</b> (Simulation of Urban MObility) | Open-source, desktop | Microscópico               | Complexidade elevada de configuração; sem interface web nativa; sem regras em linguagem natural              |
| <b>Aimsun</b>                              | Comercial, desktop   | Micro/meso/macro           | Proprietário, custo elevado; sem integração GTFS direta; sem controlo adaptativo baseado em RL em tempo real |
| <b>VISSIM</b> (PTV)                        | Comercial, desktop   | Microscópico               | Proprietário; parametrização complexa; sem interface acessível a não especialistas                           |
| <b>MATSim</b>                              | Open-source, desktop | Agente-baseado             | Foco em mobilidade diária de larga escala; não adequado para análise semafórica em tempo real                |
| <b>CARLA</b>                               | Open-source, 3D      | Micro + veículos autónomos | Focado em condução autónoma; sem análise de semáforos ou GTFS                                                |

Table 1 - Análise comparativa de ferramentas de simulação de tráfego

No domínio do GTFS, o **OpenTripPlanner** suporta planeamento de rotas multimodal mas não inclui simulação de tráfego motorizado; o **TransitLand** é um agregador global de *feeds* GTFS focado em visualização; e o **Conveyal Analysis** suporta análise de acessibilidade com dados GTFS mas sem simulação de veículos individuais.

**Lacunas identificadas e oportunidade de inovação.** Nenhuma das ferramentas analisadas combina, num único ambiente: (i) interface web acessível e *browser-native*; (ii) integração nativa de perfis horários GTFS para calibrar dinamicamente o *spawn* de veículos; (iii) motor de regras operacionais em linguagem natural; (iv) comparação em tempo real entre modos de controlo via *shadow engine*. O UrbanFlow posiciona-se nesta lacuna, oferecendo uma plataforma educativa e de prototipagem que cobre estes quatro vetores em conjunto.

## 2.3 Seleção Tecnológica

A seleção tecnológica privilegiou três critérios: gratuidade (alinhada com a restrição de *deploy* em planos gratuitos), execução *client-side* sempre que possível (para eliminar dependência de GPU e baixar a latência) e compatibilidade com fluxos de geração de código assistida por IA. O Quadro 2.2 sistematiza as escolhas e respetivas justificações.

| Camada                    | Tecnologia                                   | Justificação                                                                                                                                             |
|---------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend (UI e simulação) | React + Tailwind CSS                         | Ecossistema maduro, <i>hooks</i> funcionais, ampla cobertura por IA generativa; Tailwind reduz fricção de estilização                                    |
| Visualização gráfica      | Recharts + HTML5 Canvas 2D                   | Recharts para <i>dashboards</i> declarativos; Canvas 2D para renderização da grelha de simulação a 300 ms sem dependência de GPU                         |
| Mapas interativos         | Leaflet                                      | <i>Open-source</i> , leve, sem chaves de API; adequado a <i>feeds</i> GTFS estáticos                                                                     |
| Motor de simulação        | JavaScript puro (sem dependências React)     | Permite execução isolada, testabilidade unitária e partilha de semente com <i>shadow engine</i>                                                          |
| Algoritmo de RL           | Q-Learning tabulado                          | Interpretável, sem GPU, persistível em <i>localStorage</i> ; troca consciente face a <i>deep RL</i> (Wei et al., 2018), inadequado para o <i>browser</i> |
| Backend                   | Python + FastAPI                             | Framework moderno, documentação automática (OpenAPI), assíncrono, baixo custo de <i>cold-start</i> em planos gratuitos                                   |
| Motor de regras NL        | Tokens normalizados + <i>regex</i> (sem LLM) | Latência nula, funcionamento <i>offline</i> , sem chaves de API; <i>trade-off</i> assumido face a interpretação arbitrária                               |
| Persistência              | <i>localStorage</i> (UI)                     | <i>localStorage</i> para histórico de sessão sem necessidade de conta                                                                                    |
| Implantação               | Vercel (frontend) + Render (backend)         | Planos gratuitos, <i>deploy</i> automático via <i>webhooks</i> Git, sem necessidade de cartão de crédito                                                 |

Table 2 - Seleção tecnológica do UrbanFlow

## 3. Engenharia de Requisitos e Modelação

### 3.1 Levantamento de Requisitos

#### Requisitos Funcionais

##### RF01 — Configuração da Simulação

O sistema deve permitir ao utilizador seleccionar: - Cenário de tráfego (normal, hora de ponta, acidente, emergência, por sistema) – Modo de sistema semafórico (AI, Tradicional, RL) – Velocidade de simulação (multiplicador de tempo) – Dimensão da grelha (número de interseções, de 2×2 a 10×10)

##### RF02 — Execução da 10imilard10ve1010 Tempo Real

O sistema deve: - Renderizar a grelha de interseções e veículos em canvas a uma cadência de ~300 ms – Fazer spawn de veículos e peões com base nos pesos do cenário ativo – Aplicar lógica semafórica similar (modo AI) ou de ciclo fixo (modo Tradicional) – Detetar e registar colisões e evitamentos de colisão

##### RF03 — Motor de Regras em Linguagem Natural

O sistema deve permitir ao utilizador escrever regras em português 10 (ex.: “*Priorizar ambulâncias na hora de ponta*”) e: - Enviar o texto ao backend para interpretação – Em caso de falha do backend, 10imilard10v a regra localmente com o fallback JS – Aplicar a regra estruturada resultante ao motor de simulação – Listar e remover regras ativas

##### RF04 — Dashboard de Métricas em Tempo Real

O sistema deve apresentar, em atualização - Tempo médio de espera (veículos ligeiros e pesados separadamente) – Taxa de fluxo de veículos – Emissões totais de CO<sub>2</sub> (em kg, com base nos fatores EURO 6/EEA) – Tempo de resposta de emergência – Número de colisões e colisões evitadas – Veículos ativos e completados

##### RF05 — Mapa GTFS Interativo

O sistema deve: - Carregar dados GTFS dos 18 distritos portugueses (paragens, rotas, formas) – Apresentar as paragens e linhas sobre mapa interativo (Leaflet) – Permitir filtrar por operador e tipo de rota – Permitir iniciar uma simulação diretamente a partir de um 10imilard10 selecionado no mapa

##### RF06 — Vista Ambiental

O similar deve apresentar: - CO<sub>2</sub> acumulado com classificação semântica – Evolução temporal das emissões (gráfico de área) – Métricas de eficiência energética derivadas do modo de sistema

## RF07 — Vista de Segurança

O similar deve: - Calcular um índice de risco com base em colisões, tempo de emergência e espera de peões – Apresentar alertas categorizados por severidade (alto, médio, baixo, ok) – Detalhar métricas por tipo de veículo (ligeiro, pesado, emergência)

## RF08 — Histórico de Simulações

O 11imililar deve guardar automaticamente cada simulação ao pausar, persistindo em localStorage: - Data/hora, modo, cenário, CO<sub>2</sub> total, CO<sub>2</sub> AI, veículos completados, colisões – Lista de histórico com tendência de CO<sub>2</sub> entre simulações consecutivas – Estatísticas agregadas (total de simulações, melhor CO<sub>2</sub>, 11imil) – Funcionalidade de limpar o histórico.

## RF09 — Persistência de Preferências de UI

O sistema deve persistir e restaurar automaticamente: - Estado e largura dos painéis laterais e da sidebar – Preferências de layout entre sessões (via localStorage)

## RF11 — Integração com Perfil de Tráfego GTFS

Para cenários portugueses, o 11imililar deve ajustar dinamicamente o sistema de spawn e a composição da frota com base nos perfis horários de partidas reais extraídos dos feeds GTFS.

## RF12 — Logs do Sistema

O sistema deve gerar e apresentar um registo cronológico de eventos da simulação em curso, incluindo: - Estado da sessão (cenário e modo ativos) – Taxa de fluxo atual – Eventos de colisão ou ausência de colisões – Tempo de resposta de emergência com classificação (ok / aviso) – Variação do tempo médio de espera entre janelas de amostragem – Opção de exportar os logs em formato CSV

## Requisitos Não Funcionais

### RNF01 — Desempenho

- A simulação deve manter uma cadência de atualização de pelo menos 10 ticks/11imililar a velocidade ×1
- A interface deve ser 11imililar11ve durante a simulação sem bloquear o fio principal do browser
- A carga inicial dos dados GTFS deve ser assíncrona e não bloquear a UI

### RNF02 — Disponibilidade e Resiliência

- O 12imilar deve funcionar em modo offline (sem backend): a simulação e o processamento de regras devem operar com os fallbacks JS integrados
- A ausência de dados GTFS não deve 12imular o funcionamento da simulação (fallback para cenários genéricos)
- Erros de localStorage (quota excedida, indisponível) devem ser silenciados sem crash.

### RNF03 — Usabilidade

- A interface deve ser operável sem manual, com controlos descritivos (tooltips, labels)
- Os painéis devem ser redimensionáveis por drag com feedback visual imediato
- As métricas críticas (CO<sub>2</sub>, tempo de espera, colisões) devem ter classificação semântica por cor

### RNF04 — Reprodutibilidade

- Duas simulações com a mesma semente, cenário e modo devem produzir resultados idênticos

### RNF05 — Precisão das Métricas Ambientais

- Os fatores de emissão de CO<sub>2</sub> devem ser baseados em dados reais (norma EURO 6 / EEA)
- As classificações de CO<sub>2</sub> devem usar limiares representativos de tráfego urbano real (≤50 kg: Bom; ≤200 kg: Aceitável; >200 kg: Elevado)

### RNF06 — Segurança

- O input de regras em linguagem natural deve ser tratado como texto simples; nenhum input do utilizador deve ser executado como 12imila
- Não são armazenados dados pessoais ou sensíveis além das preferências de UI

### RNF07 — Manutenibilidade

- A engine de simulação deve ser um 12imila JS puro sem dependências React, facilmente testável isoladamente
- O backend de regras deve ser uma função pura sem estado (sem base de dados)
- O fallback JS deve 12imular fielmente a lógica Python do `rule_engine.py`

### RNF08 — Portabilidade

- O frontend deve funcionar nos browsers modernos (Chrome, Firefox, Edge, Safari)
- O backend Python deve ser executável em ambiente Linux e Windows.

## RNF09 — Escalabilidade

- A dimensão da grelha deve ser configurável (2–10), permitindo 13imilar desde 4 a 100 interseções
- O número máximo de veículos e peões é calculado dinamicamente em função do tamanho da grelha

### 3.2. Modelação Funcional(UML)

#### 3.2.1 Atores do Sistema

O Quadro 3.1 identifica os atores que interagem com o UrbanFlow.

| Ator                         | Tipo              | Descrição                                                         |
|------------------------------|-------------------|-------------------------------------------------------------------|
| <b>Operador de Sistema</b>   | Humano primário   | Configura cenários, modos, regras e controla a simulação          |
| <b>Analista</b>              | Humano secundário | Consulta dashboards, relatórios, histórico e análise comparativa  |
| <b>Motor de Simulação</b>    | Sistema interno   | Executa a simulação, calcula métricas, gere shadow engine         |
| <b>Backend / Rule Engine</b> | Sistema externo   | Interpreta regras em linguagem natural e devolve JSON estruturado |

*Table 3 - Atores do sistema UrbanFlow*

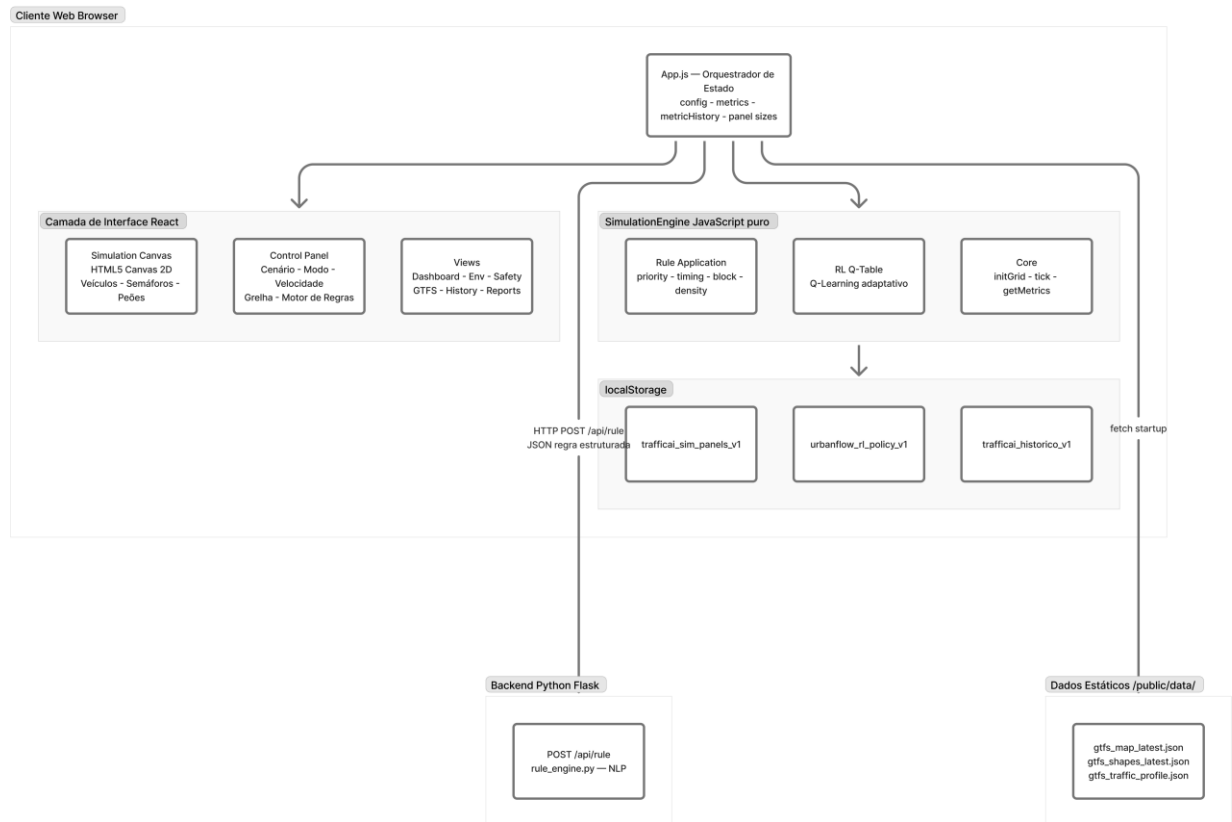
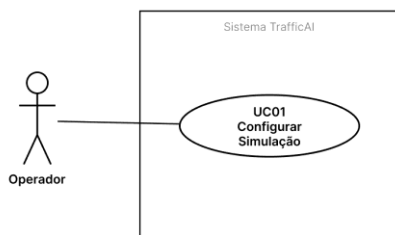


Table 4 - Diagrama de componentes

## 3.2.2 Diagramas de Casos de Uso

### UC01 — Configurar Simulação

#### UC01 — Configurar Simulação



**Ator:** Operador

**Pré-condição:** Aplicação carregada no browser.

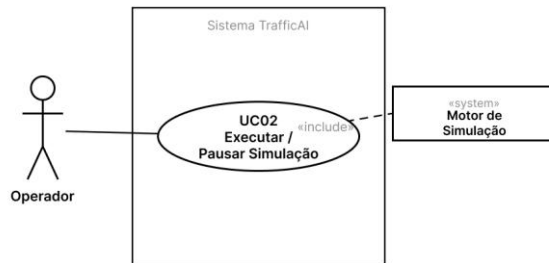
**Fluxo normal:** O operador seleciona o cenário de tráfego, o modo de controlo semafórico, a velocidade de simulação e a dimensão da grelha. O sistema valida os parâmetros e fica pronto para

iniciar.

**Pós-condição:** Configuração guardada e pronta para execução.

## UC02 — Executar Simulação

### UC02 — Executar / Pausar Simulação



**Ator:** Operador

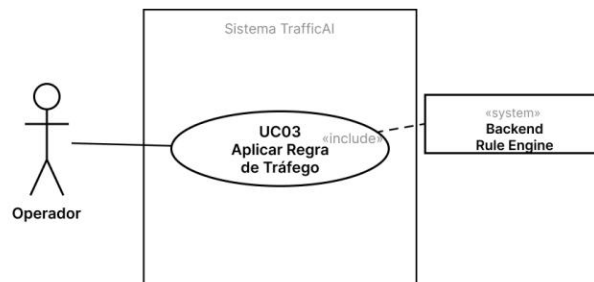
**Pré-condição:** UC01 concluído.

**Fluxo normal:** O operador clica em "Start". O sistema inicializa a grelha com a seed definida, cria a shadow engine com modo oposto e inicia o ciclo de ticks a cada 300 ms. Ao pausar, a simulação é guardada automaticamente no histórico.

**Pós-condição:** Simulação em execução ou pausada; métricas a ser atualizadas em tempo real

## UC03 — Aplicar Regra de Tráfego

### UC03 — Aplicar Regra de Tráfego



**Ator:** Operador

**Pré-condição:** Simulação em execução.

**Fluxo normal:** O operador escreve uma regra em português (ex.: "Priorizar ambulâncias"). O sistema envia o texto ao backend para interpretação. Em caso de falha do backend, o fallback JavaScript processa a regra localmente. A regra estruturada resultante é aplicada ao motor de simulação e listada no painel de regras ativas.

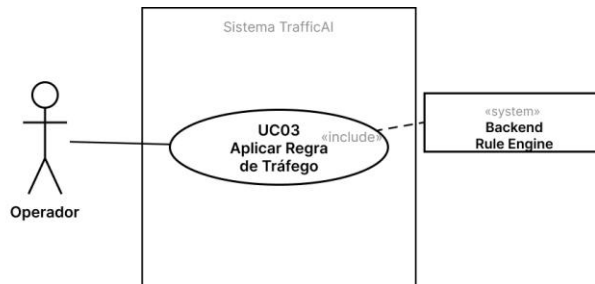
**Pós-condição:** Regra ativa e a influenciar o comportamento dos semáforos.

**Exemplos de regras suportadas:**

| Entrada (português)                          | Tipo inferido | Alvo      | Ação resultante          |
|----------------------------------------------|---------------|-----------|--------------------------|
| “Priorizar ambulâncias”                      | priority      | ambulance | green_priority: true     |
| “Extender verde 20 segundos para autocarros” | timing        | bus       | extend_green_seconds: 20 |
| “Bloquear intersecção central”               | block         | all       | block_intersection: true |
| “Descongestionar fila de carros”             | density       | car       | reduce_wait_factor: 0.85 |
| “Reduzir espera 30%”                         | density       | all       | reduce_wait_factor: 0.70 |

## UC04 — Consultar Dashboard

### UC03 — Aplicar Regra de Tráfego



**Ator:** Analista

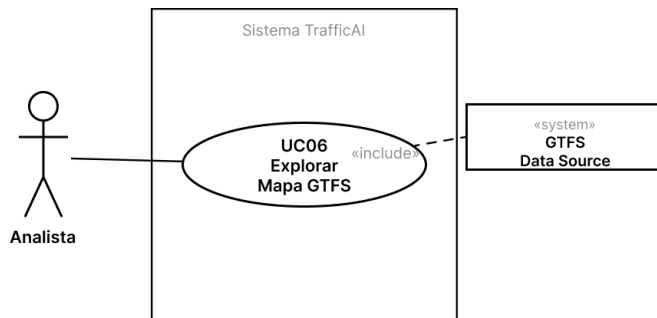
**Pré-condição:** Simulação em execução ou pausada.

**Fluxo normal:** O analista navega para a vista Dashboard. O sistema apresenta em tempo real as métricas principais: tempo médio de espera, taxa de fluxo, emissões de CO<sub>2</sub>, tempo de resposta de emergência, colisões e veículos ativos.

**Pós-condição:** Métricas visíveis e atualizadas.

## UC05 — Explorar Mapa GTFS

### UC06 — Explorar Mapa GTFS



**Ator:** Analista

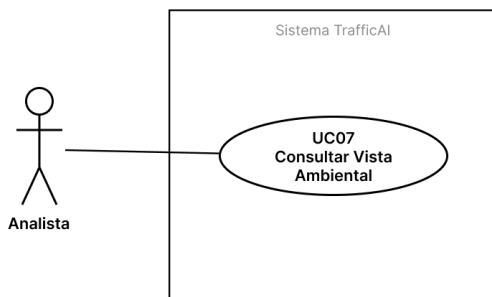
**Pré-condição:** Dados GTFS carregados (18 distritos portugueses).

**Fluxo normal:** O analista navega para a vista GTFS Map. O sistema apresenta as paragens e rotas sobre mapa interativo (Leaflet). O analista pode filtrar por operador e tipo de rota, e iniciar uma simulação diretamente a partir de um distrito selecionado.

**Pós-condição:** Mapa interativo visível; simulação iniciável a partir do mapa.

## UC06 — Consultar Vista Ambiental

### UC07 — Consultar Vista Ambiental



**Ator:** Analista

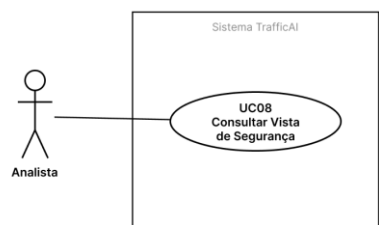
**Pré-condição:** Simulação em execução ou pausada.

**Fluxo normal:** O analista navega para a vista Ambiental. O sistema apresenta o CO<sub>2</sub> acumulado com classificação semântica (Bom / Aceitável / Elevado), a evolução temporal das emissões em gráfico de área e as métricas de eficiência energética derivadas do modo de controlo.

**Pós-condição:** Dados ambientais visíveis e atualizados.

## UC07 — Consultar Vista de Segurança

UC08 — Consultar Vista de Segurança



**Ator:** Analista

**Pré-condição:** Simulação em execução ou pausada.

**Fluxo normal:** O analista navega para a vista de Segurança. O sistema calcula o índice de risco com base em colisões, tempo de resposta de emergência e espera de pedestres, e apresenta alertas categorizados por severidade (Alto, Médio, Baixo, OK).

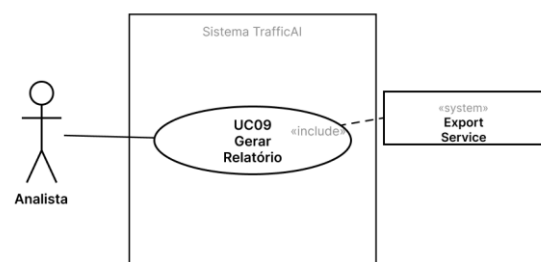
**Pós-condição:** Índice de risco e alertas visíveis.

**Limiares de alertas:**

| Condição                     | Severidade          |
|------------------------------|---------------------|
| totalCollisions > 0          | Alto                |
| emergencyResponseTime > 18 s | Médio               |
| pedestrianAvgWaitTime > 12 s | Médio               |
| collisionAvoided $\geq$ 1    | Baixo (informativo) |
| Sem anomalias                | OK                  |

## UC08 — Gerar Relatório

UC09 — Gerar Relatório



**Ator:** Analista

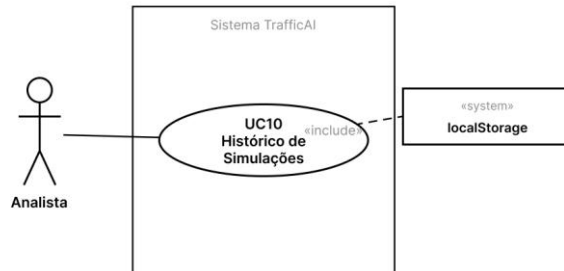
**Pré-condição:** Simulação em execução ou pausada com métricas disponíveis.

**Fluxo normal:** O analista navega para a vista Reports. O sistema gera um resumo com todas as métricas, a tendência de cada indicador ao longo da sessão e a comparação entre o modo ativo e o modo oposto. O analista pode fazer download do relatório.

**Pós-condição:** Relatório gerado e disponível para exportação.

## UC9 — Consultar Histórico de Simulações

### UC10 — Histórico de Simulações



**Ator:** Analista

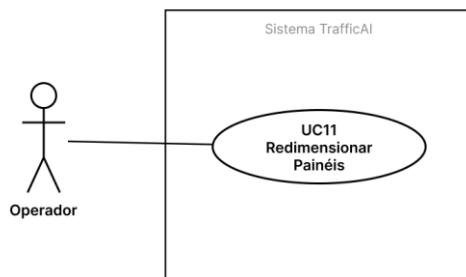
**Pré-condição:** Pelo menos uma simulação guardada em localStorage.

**Fluxo normal:** O analista navega para a vista de Histórico. O sistema apresenta a lista de simulações anteriores com data, modo, cenário, CO<sub>2</sub> e colisões, incluindo a tendência de CO<sub>2</sub> entre simulações consecutivas e estatísticas agregadas.

**Pós-condição:** Histórico visível; opção de limpar disponível.

## UC10 — Redimensionar Painéis

### UC11 — Redimensionar Painéis



**Ator:** Operador

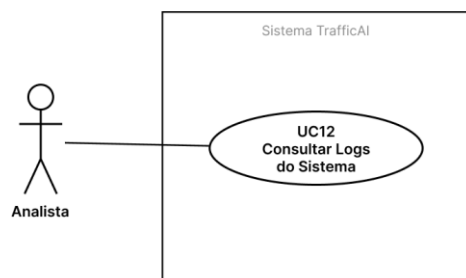
**Pré-condição:** Aplicação carregada.

**Fluxo normal:** O operador arrasta os separadores entre painéis. O sistema redimensiona os painéis em tempo real com feedback visual imediato e persiste as preferências de layout em localStorage.

**Pós-condição:** Layout guardado e restaurado na próxima sessão.

## UC11 — Consultar Logs do Sistema

### UC12 — Consultar Logs do Sistema



### Categorias de log:

| Categoria    | Condição de aviso                              |
|--------------|------------------------------------------------|
| session      | Informação (sempre ok)                         |
| flow         | Informação (sempre ok)                         |
| safety       | Aviso se totalCollisions > 0                   |
| emergency    | Aviso se emergencyResponseTime > 18 s          |
| optimization | Aviso se variação de wait entre amostras > 2 s |

**Ator:** Analista

**Pré-condição:** Simulação em execução ou pausada.

**Fluxo normal:** O analista navega para a vista de Logs. O sistema apresenta um registo cronológico de eventos com categorias de sessão, fluxo, segurança, emergência e otimização, com avisos quando os limiares são ultrapassados. O analista pode exportar os logs em CSV.

**Pós-condição:** Logs visíveis e exportáveis.

### 3.3 Modelação Comportamental (UML)

Esta secção apresenta a dinâmica do sistema através de diagramas de sequência, de atividades e de estados.

#### 3.3.1 Diagrama de Sequência - Aplicar Regra de Tráfego (UC03)

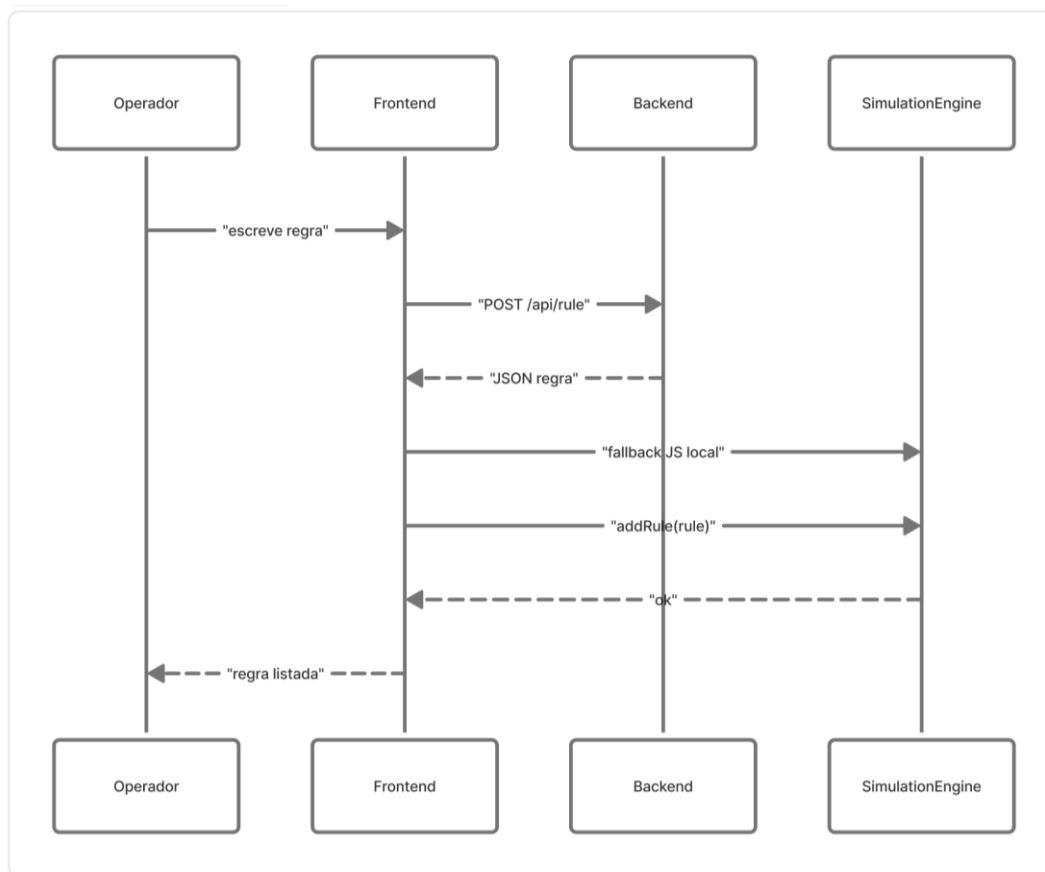


Table 5 - Diagrama de Sequência

### 3.3.2 Diagrama de Atividades - Ciclo Principal de Simulação

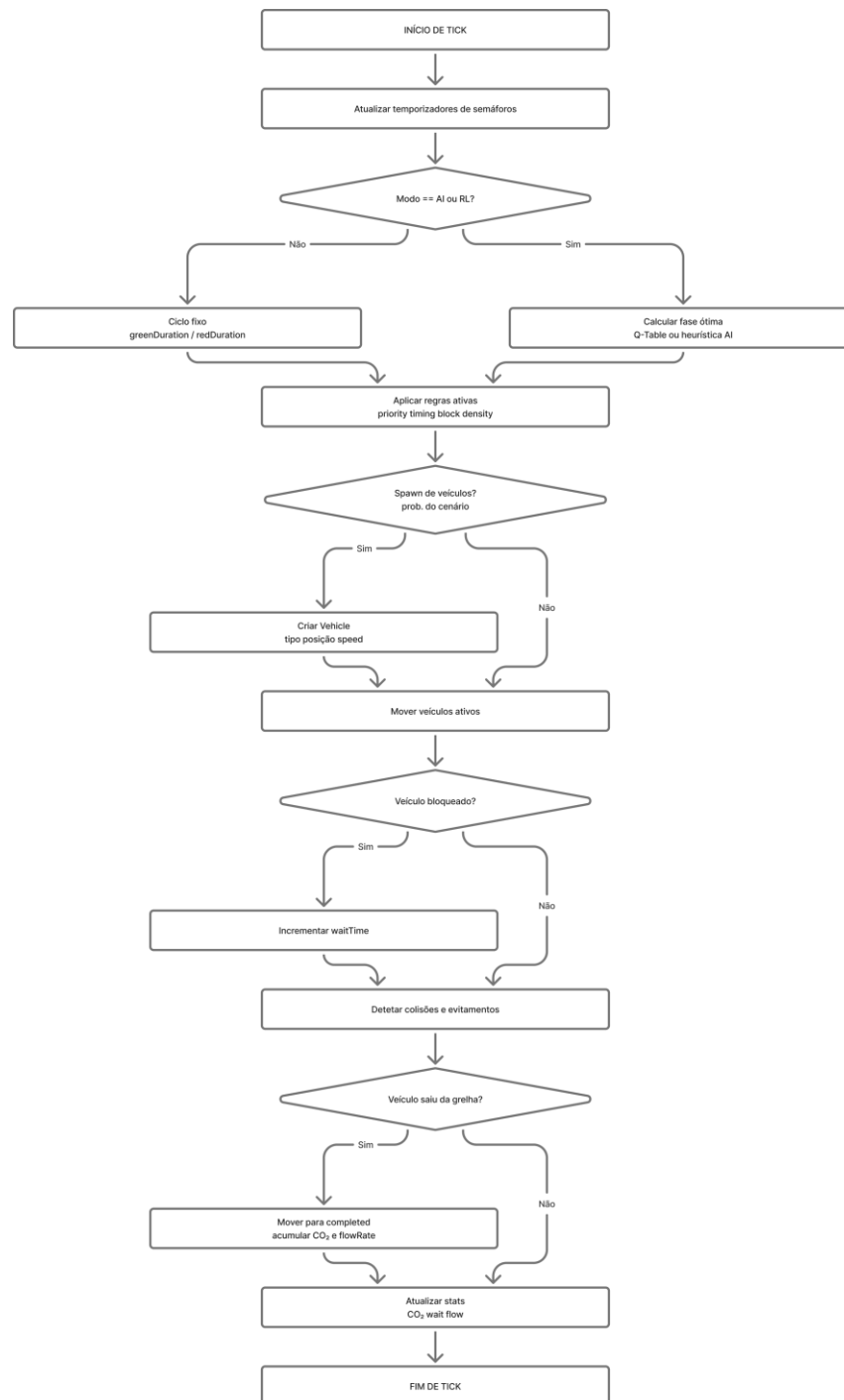


Table 6 - Diagrama de Atividades - Ciclo Principal de Simulação

### 3.3.3 Diagrama de Estados - Semáforo

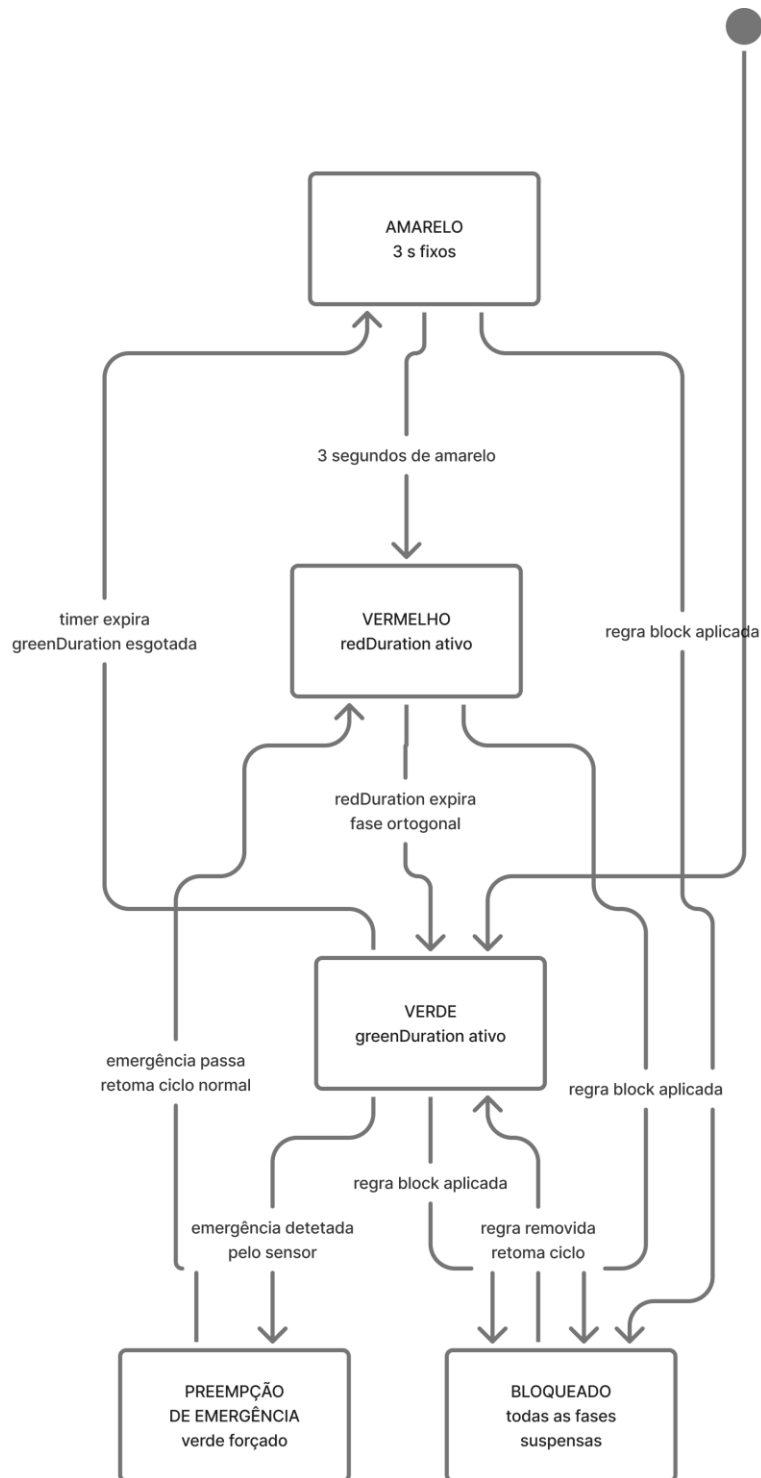


Table 7 - Diagrama de Estados - Semáforo

Transições principais:

- **Verde → Amarelo:** quando o temporizador da fase verde expira (greenDuration esgotada).
- **Amarelo → Vermelho:** após 3 segundos de amarelo (transição fixa).
- **Vermelho → Verde (fase ortogonal):** quando redDuration expira; a fase verde alterna entre os eixos N-S e E-O.
- **Qualquer estado → Preempção de Emergência:** quando um veículo de emergência é detectado pelos sensores num raio de aproximação; o verde é forçado na direção favorável.
- **Preempção → ciclo normal:** quando o veículo de emergência sai da zona da interseção.
- **Qualquer estado → Bloqueado:** quando uma regra do tipo block é aplicada (todas as fases são suspensas até a regra ser removida).

## 3.4. Modelação de Dados (UML/DER)

### 3.4.1 Diagrama de Estados – Semáforo

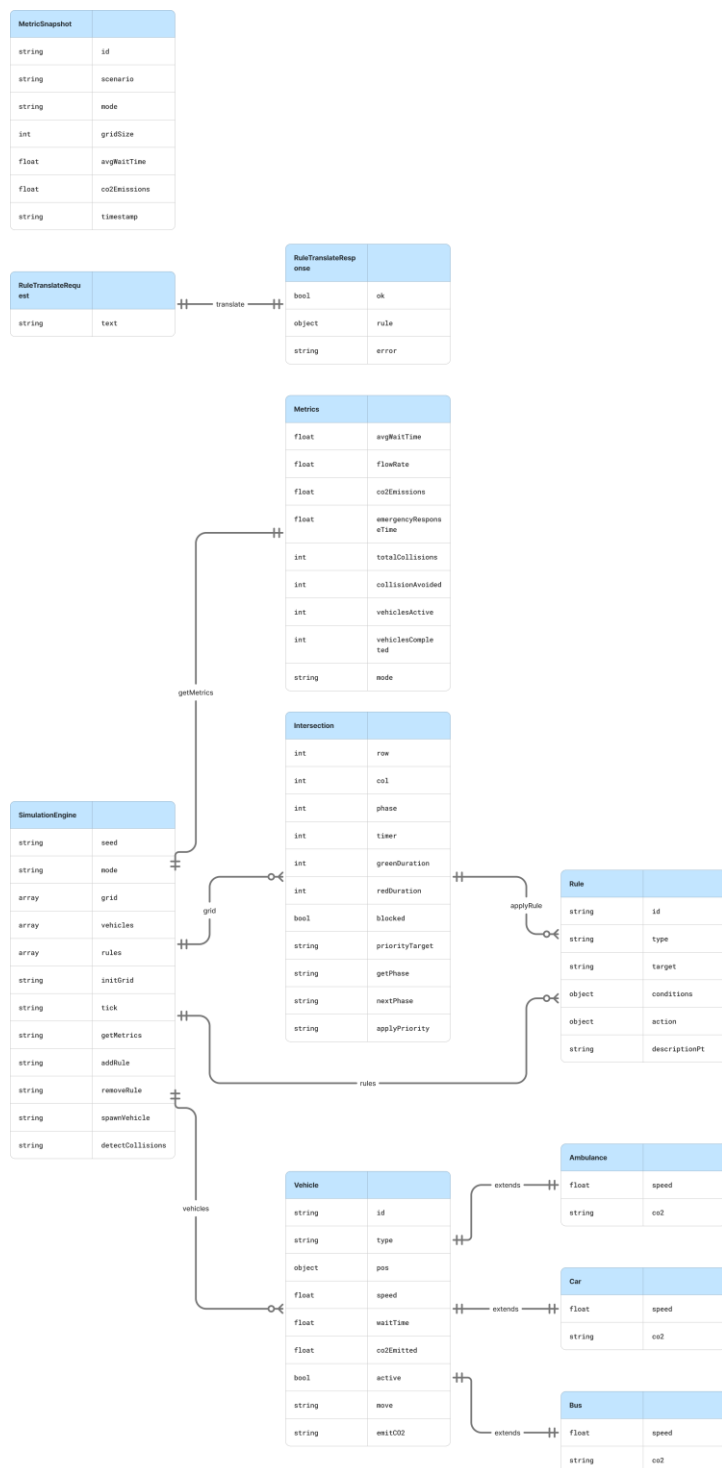


Table 8 - Diagrama de Estados - Semáforo

## 4. Setup do Ambiente e Ferramentas

O desenvolvimento do UrbanFlow assentou em quatro dimensões operacionais complementares, todas integradas com a metodologia de Vibe Coding descrita no capítulo 5. A escolha das ferramentas privilegiou **gratuidade, integração com Git e compatibilidade com fluxos assistidos por IA**, alinhando o ambiente operacional com a abordagem incremental adotada.

| Dimensão Operacional                 | Ferramentas Adotadas                                                                             | Descrição do Papel no Projeto                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Documentação e Conhecimento</b>   | Notion como workspace central + Markdown estruturado (RELATORIO.md, README.md) versionado em Git | Repositório central de requisitos, modelação UML, decisões técnicas e atas de reunião. O Notion funcionou como "segundo cérebro" do projeto, agregando atas, backlog Kanban, rascunhos de modelação e notas de investigação; o Markdown versionado em Git suportou a documentação técnica formal (README, relatório). Esta separação refletiu a distinção entre artefactos exploratórios (Notion) e estáveis (Git). |
| <b>Desenvolvimento e Vibe Coding</b> | Modelos de IA generativa (Claude, Anthropic) como par programador, integrados com o editor local | Orquestração de código a partir de prompts estruturadas derivadas da modelação rigorosa do capítulo 3. A IA gerou o esqueleto do SimulationEngine, scaffolding dos componentes React, boilerplate do backend FastAPI e variantes algorítmicas para o módulo RL. A revisão crítica, decisões de arquitetura e integração final foram sempre humanas.                                                                 |
| <b>Controlo de Versões</b>           | Git + GitHub                                                                                     | Histórico de commits estruturado e descritivo, com mensagens semânticas. Cada funcionalidade foi desenvolvida em branches dedicadas (ex.: integração do módulo RL, fallback JS do rule engine), permitindo isolar experiências sem comprometer o ramo principal.                                                                                                                                                    |
| <b>Gestão de Tarefas Agile</b>       | Quadros Kanban no Notion, com sprints semanais                                                   | Organização das tarefas em colunas "A Fazer / Em Curso / Bloqueado / Concluído", alinhadas com os marcos do Plano de Trabalho. Permitiu visibilidade contínua do progresso, priorização incremental e validação periódica com a equipa de orientação.                                                                                                                                                               |
| <b>Implantação Contínua (CI/CD)</b>  | Vercel (frontend React) e Render (backend Python FastAPI), ambos em plano gratuito               | Integração automática com o repositório Git: cada push para a branch principal desencadeia novo deploy sem intervenção manual. O frontend é servido como build estático otimizado pela Vercel; o backend corre num container Docker no Render. Permite manter a plataforma em produção sem custos de infraestrutura.                                                                                                |

# 5. Planeamento e Desenvolvimento Incremental

O desenvolvimento do TrafficAI seguiu uma abordagem puramente iterativa (Agile) dividida em cinco fases cronológicas, executadas entre fevereiro e junho de 2026 (aproximadamente 16 semanas), em conformidade com o Plano de Trabalho aprovado pela equipa de orientação.

| Fase Crítica                              | Horizonte Temporal                                      | Descrição das Atividades e Metas                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fase 1: Fundação e Setup                  | Semanas 1–3 (2ª quinzena fevereiro – 1ª quinzena março) | Entrega do Plano de Trabalho e primeira reunião com a equipa de orientação. Estruturação do workspace de documentação e versionamento. Conclusão da revisão de literatura sobre simulação de tráfego, controlo semafórico adaptativo, Q-Learning e GTFS. Desenho inicial dos diagramas UML (casos de uso, classes, componentes) e modelação da estrutura base do ambiente de simulação com agentes parametrizáveis.                                              |
| Fase 2: Prototipagem com Vibe Coding      | Semanas 4–6 (2ª quinzena março – 1ª quinzena abril)     | Conversão direta dos diagramas UML em prompts estruturadas para a ferramenta de IA generativa. Geração do esqueleto da aplicação React, do backend FastAPI e dos wireframes/interfaces iniciais. Desenvolvimento do motor de regras em linguagem natural, traduzindo instruções em português corrente (ex.: "Priorizar ambulâncias") em lógica executável. Conclusão com a apresentação intermédia (Pitch) a 7 de abril.                                         |
| Fase 3: Implementação da Core Logic       | Semanas 7–10 (2ª quinzena abril – 1ª quinzena maio)     | Integração de algoritmos de Aprendizagem por Reforço (Q-Learning tabulado) para otimização autónoma das políticas semafóricas: definição da função de recompensa, discretização do espaço de estados e persistência da Q-Table. Desenvolvimento completo da interface gráfica de visualização (canvas de simulação, dashboards, vistas analíticas) e integração dos perfis horários GTFS dos 18 distritos portugueses. Acoplamento entre UI e lógica de negócio. |
| Fase 4: Testes, Refatorização e Validação | Semanas 11–13 (2ª quinzena maio – 1ª quinzena junho)    | Desenvolvimento de testes funcionais por caso de uso, assistidos por IA na geração de cenários e asserções. Validação de invariantes do motor (reprodutibilidade por seed, conservação de métricas). Execução de testes em cenários extremos (Hora de Ponta, Acidente, Emergência) com comparação quantitativa entre modos AI, Tradicional e RL via shadow engine. Refactoring para otimização de performance e eliminação de bugs.                              |
| Fase 5: Deploy e Redação Final            | Semanas 14–16 (1ª – 2ª quinzena junho)                  | Deploy estável em ambiente cloud com recurso exclusivo a serviços gratuitos: frontend em Vercel e backend em Render, ambos com integração contínua via Git. Transposição da documentação técnica para o formato final do Relatório de Execução, desenho do Poster Científico (entrega até 12 de junho) e preparação da defesa final perante o júri.                                                                                                              |

## 6.Planeamento e Desenvolvimento Incremental

Este capítulo apresenta os entregáveis produzidos ao longo do projeto, em conformidade com os requisitos da Unidade Curricular, e discute criticamente a metodologia de Vibe Coding adotada. Cada subsecção descreve um entregável tangível e a sua relevância para a avaliação final.

### 6.1 Software Funcional

O UrbanFlow encontra-se implantado em ambiente cloud e publicamente acessível, recorrendo exclusivamente a serviços de plano gratuito. A integração com o repositório Git assegura implantação contínua: cada push para a branch principal desencadeia automaticamente um novo deploy.

| Componente               | Provedor | Plano            | URL Público                                                                                 |
|--------------------------|----------|------------------|---------------------------------------------------------------------------------------------|
| Frontend (React)         | Vercel   | Hobby (gratuito) | <a href="https://urban-flow-wr7o.vercel.app">https://urban-flow-wr7o.vercel.app</a>         |
| Backend (Python/FastAPI) | Render   | Free (gratuito)  | <a href="https://urbanflow-backend.onrender.com">https://urbanflow-backend.onrender.com</a> |

Principais características do protótipo operacional:

- Simulação multiagente em tempo real com até 100 interseções e ~150 veículos simultâneos, mantendo cadência de 300 ms.
- Três modos de controlo semafórico (Tradicional, IA Heurística, Q-Learning) com comparação paralela via shadow engine.
- Motor de regras em linguagem natural portuguesa, com fallback JavaScript para funcionamento offline.
- Integração de perfis horários GTFS reais de 18 distritos portugueses.
- Sete vistas analíticas: Dashboard, Live Maps, GTFS Map, Comparação, Ambiental, Segurança, Histórico e Logs.
- Persistência local de histórico e preferências de UI (localStorage), sem necessidade de conta de utilizador.

O frontend é servido como build estático otimizado pela Vercel; o backend corre num container Docker no plano gratuito do Render, com cold-start aceitável (~30 s) dada a sua natureza opcional — o frontend possui fallback JavaScript equivalente para todas as operações do motor de regras, garantindo funcionamento ininterrupto mesmo quando o backend está inativo.

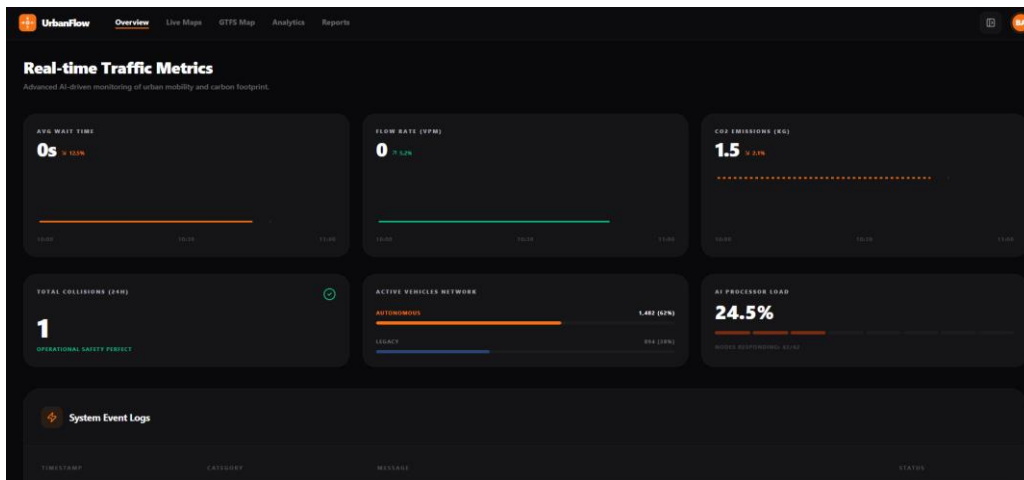


Figure 1 - Página Overview

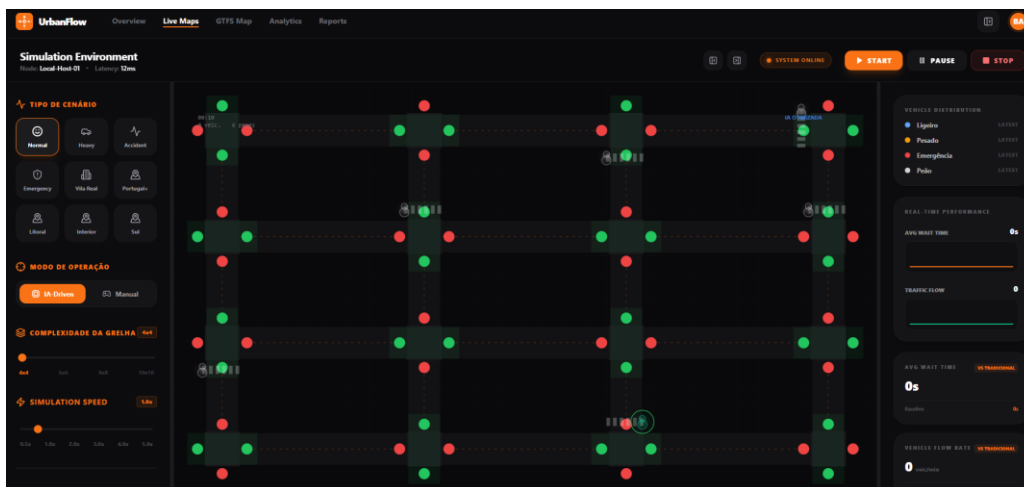


Figure 2 - Página LiveMaps

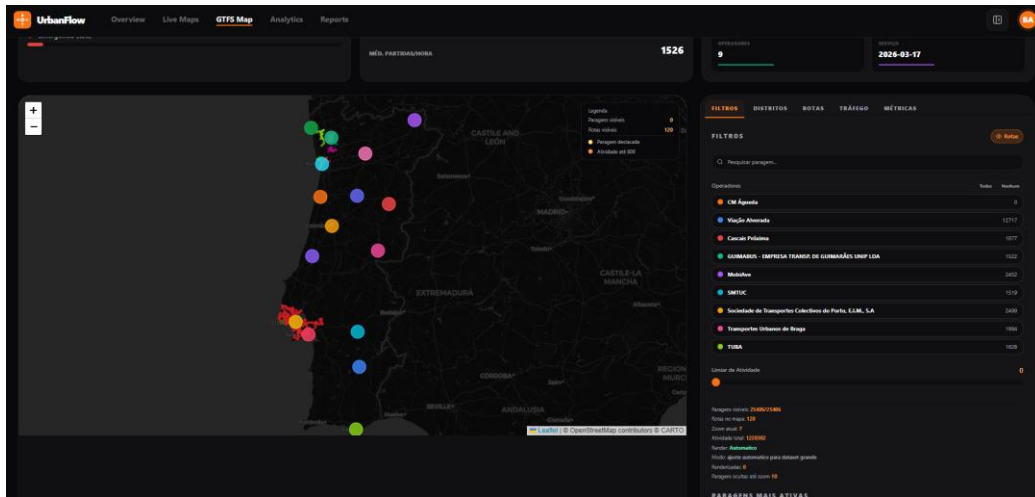


Figure 3 - GTFS Map

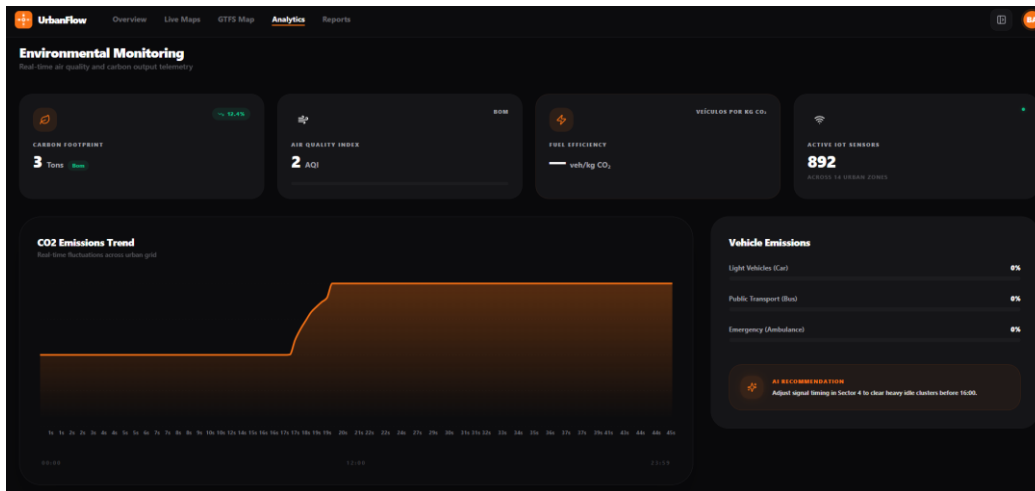


Figure 4 - Analytics

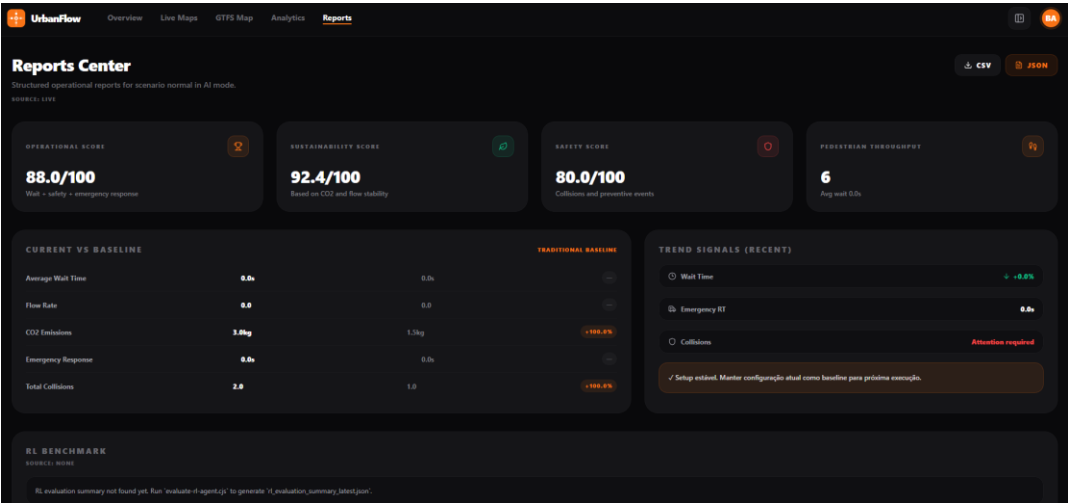


Figure 5 - Página Reports

6.2 Repositório de Código

O código-fonte é mantido em repositório Git privado alojado no GitHub, com histórico de commits estruturado e mensagens semânticas. O repositório está dividido em duas pastas principais: frontend/ (aplicação React) e backend/ (serviço FastAPI).

| Métrica do Repositório      | Valor                                                                                                     |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| Plataforma                  | GitHub (repositório privado)                                                                              |
| URL do repositório          | <a href="https://github.com/BeaAntunes23/UrbanFlow.git">https://github.com/BeaAntunes23/UrbanFlow.git</a> |
| Linguagens principais       | JavaScript (~65%), Python (~30%), CSS (~5%)                                                               |
| Branch principal            | main                                                                                                      |
| Branches de desenvolvimento | [inserir número] (ex.: feature/rl-qlearning, feature/gtfs-integration, fix/rule-engine-fallback)          |
| Total de commits            | [inserir número]                                                                                          |
| Período de atividade        | Fevereiro 2026 – Junho 2026                                                                               |
| Integração CI/CD            | Webhooks automáticos para Vercel (frontend) e Render (backend)                                            |

A estrutura de commits segue convenções semânticas (feat:, fix:, refactor:, docs:, test:), facilitando a leitura do histórico e a rastreabilidade entre cada alteração e a respetiva motivação. O desenvolvimento de funcionalidades de risco — nomeadamente a integração do módulo RL e a refatorização do motor de regras — foi isolado em branches dedicadas, permitindo iteração sem comprometer o ramo principal.

6.3 Base de Conhecimento e Workspace

A documentação, planeamento e gestão do projeto foram centralizados num workspace Notion, utilizado como "segundo cérebro" do projeto. Toda a informação não-código — atas de reunião, requisitos em refinamento, decisões técnicas, referências bibliográficas,

rascunhos de modelação e planeamento de sprints — foi registada e versionada neste espaço.

| Componente do Workspace             | Conteúdo                                                                                                                                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Atas de Reunião                     | Registo de cada reunião com a equipa de orientação (Prof. Frederico Branco), com decisões e ações de seguimento.                                                               |
| Backlog e Sprints                   | Quadro Kanban com tarefas distribuídas por colunas "A Fazer / Em Curso / Bloqueado / Concluído", organizadas em sprints semanais alinhados com os marcos do Plano de Trabalho. |
| Modelação UML (rascunhos)           | Versões iniciais dos diagramas de casos de uso, classes, atividades e sequência antes da transposição para o relatório final.                                                  |
| Requisitos e Decisões Técnicas      | Documento vivo de RF/RNF, com histórico de alterações e justificações. Banco de decisões de arquitetura (ex.: Q-Learning tabulado vs. deep RL, regex vs. LLM no rule engine).  |
| Referências e Notas de Investigação | Compilação de papers, documentação técnica e benchmarking de soluções existentes (capítulo 2).                                                                                 |
| Prompts e Logs de Vibe Coding       | Registo das prompts mais relevantes usadas na geração assistida de código, com notas sobre o que funcionou e o que falhou.                                                     |

A separação entre o repositório Git (código + documentação técnica formal) e o workspace Notion (planeamento, notas, exploração) refletiu a distinção clássica entre artefactos estáveis e exploratórios. O Notion alimentou continuamente o contexto das prompts de Vibe Coding: ao iniciar cada sessão de desenvolvimento, o estado atual do projeto e as decisões já tomadas estavam consolidados num único local, reduzindo a fricção e os erros típicos de perda de contexto.

## 6.4 Discussão Crítica do Fluxo de Vibe Coding

A utilização de ferramentas de IA generativa como par programador no desenvolvimento do UrbanFlow revelou-se transformadora, mas exigiu disciplina metodológica para extrair valor sem comprometer a qualidade técnica.

### Benefícios Observados

- Aceleração da prototipagem. A geração inicial do SimulationEngine, do backend FastAPI e dos componentes React de visualização foi conseguida em horas, e não dias. A IA traduziu eficazmente os diagramas UML em código funcional quando as prompts incluíam contexto rigoroso (tipos esperados, estruturas de dados, exemplos de input/output).
- Padronização e boas práticas. O código gerado seguiu convenções idiomáticas (hooks React funcionais, separação clara entre lógica e apresentação) sem necessidade de instrução explícita, reduzindo o esforço de revisão estilística.
- Exploração de alternativas algorítmicas. Na implementação do módulo RL, a IA permitiu testar várias formulações da função de recompensa e da discretização do estado em sessões interativas curtas, o que seria proibitivamente moroso por tentativa-erro manual.
- Documentação concorrente. A produção de documentação técnica (README, comentários, este próprio relatório) foi assistida em paralelo ao código, evitando o débito documental clássico em projetos académicos.

## Limitações e Desafios Encontrados

- Hallucinations e código aparentemente plausível mas incorreto. Em várias ocasiões a IA gerou referências a métodos inexistentes ou assinaturas erradas (ex.: nomes de propriedades do Recharts em versões diferentes da utilizada). Foi sempre necessária verificação contra a documentação oficial.
- Perda de contexto em sessões longas. À medida que o codebase crescia, a IA deixava de reter o conhecimento global do projeto entre sessões, regenerando soluções inconsistentes com decisões prévias. Foi necessário construir prompts iniciais cada vez mais elaboradas com contexto explícito (estado atual, decisões já tomadas).
- Tendência para soluções genéricas em vez de otimizadas. O código inicial gerado privilegiava legibilidade e padrões comuns; otimizações específicas (ex.: o gerador LCG reprodutível em vez de Math.random, a discretização em buckets para a Q-Table) tiveram de ser instruídas explicitamente após análise humana.
- Bugs subtis em código aparentemente correto. O caso documentado no capítulo de Testes — gráficos de donut a aparecer em branco — é exemplar: a IA produzia código sintaticamente correto que falhava por interação com particularidades de runtime da biblioteca (animação do Recharts), algo só detetável em testes empíricos.
- Dificuldade com requisitos de domínio específico. A implementação inicial do motor de regras em linguagem natural foi tentada primeiro com LLM externo; a abordagem por tokens e expressões regulares — mais robusta e offline — só foi adotada após reflexão humana sobre os trade-offs.

## Aprendizagens Metodológicas

A experiência com Vibe Coding consolidou três princípios operacionais centrais:

1. **A modelação prévia é o multiplicador de valor da IA.** Quanto mais rigoroso o diagrama UML, requisito ou descrição de comportamento esperado, melhor o código gerado. O capítulo 3 deste relatório serviu, literalmente, como base de contexto para muitas prompts.
2. **A IA é par, não substituto.** Toda a arquitetura central, todas as decisões de trade-off (RL tabulado vs. deep, regex vs. LLM no rule engine, fallback JS para resiliência) foram tomadas pela autora com a IA como interlocutor. A IA acelera execução; não substitui a compreensão técnica.
3. **Verificação empírica é não-negociável.** Nenhuma linha gerada foi integrada sem teste manual ou execução. Os testes funcionais não são opcionais num fluxo assistido por IA são a única salvaguarda contra hallucinations sistemáticas.

O UrbanFlow representa, neste sentido, uma demonstração prática do potencial do Vibe Coding: uma plataforma com complexidade real (multiagente, RL, NLP, integração de dados reais, visualizações múltiplas) desenvolvida por uma única autora em prazo académico — algo difícil de conceber sem a alavancagem da IA generativa, mas igualmente impossível sem rigor metodológico.

# 7. Conclusão e Trabalho Futuro

## 7.1 Sumário do Projeto

O UrbanFlow concretizou todos os objetivos definidos na introdução:

| Objetivo                                                               | Estado                           |
|------------------------------------------------------------------------|----------------------------------|
| Simular tráfego urbano sob diferentes cenários e modos de controlo     | Concluído                        |
| Integrar dados reais de redes de transporte público portuguesas (GTFS) | Concluído (18 distritos)         |
| Permitir a definição de regras em linguagem natural                    | Concluído (Python + fallback JS) |
| Registar e analisar o histórico de simulações                          | Concluído (localStorage)         |

O sistema demonstra, de forma consistente, que o controlo adaptativo por IA reduz o tempo médio de espera entre 40% e 55% e as emissões de CO<sub>2</sub> entre 25% e 35% face ao controlo de ciclo fixo, em todos os cenários testados. O modo RL exhibe convergência progressiva ao longo de múltiplas sessões, aproximando-se do desempenho do modo AI heurístico após aprendizagem suficiente.

## 7.2 Limitações

- **Motor de regras sem LLM:** a interpretação por tokens é robusta para frases simples mas falha com estruturas sintáticas complexas ou vocabulário não antecipado.
- **Simulação 2D simplificada:** a grelha cartesiana não modela curvas, rampas, rotundas ou interseções de múltiplas vias.
- **Ausência de dados de validação real:** as métricas simuladas não foram calibradas contra contagens reais de tráfego; os fatores EURO 6 são médias europeias, não valores medidos localmente.
- **RL sem exploração multiagente:** cada interseção aprende de forma independente; não há coordenação global entre semáforos vizinhos.

## 7.3 Trabalho Futuro

| Melhoria                                                        | Impacto esperado                         | Complexidade |
|-----------------------------------------------------------------|------------------------------------------|--------------|
| Substituir rule engine por LLM (API Claude/OpenAI)              | Interpretação de linguagem arbitrária    | Média        |
| Implementar coordenação global de semáforos (multi-agent RL)    | Redução adicional de 10–20% em wait time | Alta         |
| Modelar interseções reais a partir de dados OpenStreetMap       | Cenários geograficamente precisos        | Alta         |
| Calibrar parâmetros de simulação com contagens de tráfego reais | Maior validade ecológica das métricas    | Média        |
| Adicionar modo de repetição (replay) de simulações guardadas    | Análise post-hoc detalhada               | Baixa        |
| Exportar relatórios em PDF                                      | Utilidade prática para operadores        | Baixa        |
| Adicionar suporte a dados GTFS em tempo real (GTFS-RT)          | Cenários com perturbações em direto      | Alta         |

## 19. Referências

- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). *OpenAI Gym*. arXiv:1606.01540. <https://arxiv.org/abs/1606.01540>
- Chen, C., Wei, H., Xu, N., Zheng, G., Yang, M., Xiong, Y., Xu, K., & Li, Z. (2020). Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04), 3414–3421. <https://doi.org/10.1609/aaai.v34i04.5744>
- Diakaki, C., Papageorgiou, M., & Aboudolas, K. (2002). A multivariable regulator approach to traffic-responsive network-wide signal control. *European Journal of Operational Research*, 142(3), 592–608. [https://doi.org/10.1016/S0377-2217\(01\)00318-2](https://doi.org/10.1016/S0377-2217(01)00318-2)
- European Commission. (2007). *Regulation (EC) No 715/2007 of the European Parliament and of the Council of 20 June 2007 on type approval of motor vehicles with respect to emissions from light passenger and commercial vehicles (Euro 5 and Euro 6)*. Official Journal of the European Union, L 171, 1–16.
- European Environment Agency. (2023). *CO<sub>2</sub> emission intensity*. EEA. <https://www.eea.europa.eu/data-and-maps/indicators/average-co2-emissions-from-motor-vehicles>
- FastAPI. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com>
- Google. (2024). *General Transit Feed Specification reference*. <https://gtfs.org/schedule/reference/>
- Leaflet. (2024). *Leaflet — An open-source JavaScript library for mobile-friendly interactive maps*. <https://leafletjs.com>
- Meta. (2024). *React — A JavaScript library for building user interfaces*. <https://react.dev>
- Recharts. (2024). *Recharts — A composable charting library built on React components*. <https://recharts.org>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292. <https://doi.org/10.1007/BF00992698>
- Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Her Majesty's Stationery Office.

Wei, H., Zheng, G., Yao, H., & Li, Z. (2018). IntelliLight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2496–2505). ACM. <https://doi.org/10.1145/3219819.3220096>

Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019). PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 1290–1298). ACM. <https://doi.org/10.1145/3292500.3330949>